

Test Report

Overview

This test report describes the testing completed for my IGVC perception/object detection work in CS 4910 Senior Design.

My work focused on:

- collecting image/video data
- labeling the dataset
- training a YOLOv8 object detection model
- testing the trained model on recorded video data
- comparing model formats including `.pt`, `.onnx`, and TensorRT `.engine`

The model was trained to detect IGVC-related objects such as:

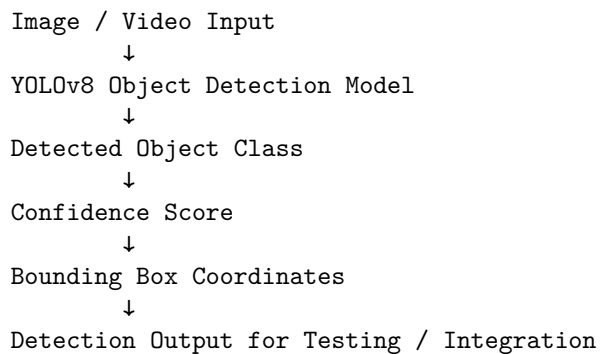
- barrels
- potholes
- signs

The final submission instructions require the test report to include metamorphic testing and mutation testing. Unit testing is not included because the final submission document says unit testing was already due earlier and does not need to be included in the final submission.

System Under Test

The system under test is the YOLOv8 object detection pipeline used for the perception portion of the IGVC project.

The general pipeline is:



The full system integration was handled by other team members. My testing focused on the object detection model and its output.

1. Metamorphic Testing

Metamorphic Relation 1: Brightness Change

Metamorphic Relation If the brightness of an image is changed slightly, the YOLOv8 model should still detect the same main object classes.

This relation is important because the IGVC vehicle may operate outdoors under different lighting conditions, such as sunlight, shade, or overcast weather.

Unit / Component Tested YOLOv8 object detection model.

Test Cases

Test Case	Description	Expected Result
MT1-A	Run detection on the original image/frame.	Model detects visible IGVC objects.
MT1-B	Run detection on the same image/frame after brightness adjustment.	Model should still detect the same main object class.

Test Script Test script: `tests/metamorphic_tests/test_brightness.py`, lines 1-32

```
from ultralytics import YOLO
import cv2

model = YOLO("best.pt")

image_path = "test_image.jpg"
image = cv2.imread(image_path)

# Test Case MT1-A: Original image
original_results = model(image)

# Test Case MT1-B: Brightness-adjusted image
bright_image = cv2.convertScaleAbs(image, alpha=1.1, beta=20)
bright_results = model(bright_image)

print("Original Image Results:")
for result in original_results:
    print(result.bboxes)

print("Brightness Adjusted Image Results:")
for result in bright_results:
    print(result.bboxes)
```

Result The brightness-adjusted input should still detect the same main visible objects. Small differences in confidence score are acceptable because lighting changes can affect model confidence.

Metamorphic Relation 2: Image Resize

Metamorphic Relation If an image is resized while keeping the same visual content, the YOLOv8 model should still detect the same main object classes.

This relation is important because image/video frames may be processed at different resolutions during testing or deployment.

Unit / Component Tested YOLOv8 object detection model.

Test Cases

Test Case	Description	Expected Result
MT2-A	Run detection on the original image/frame.	Model detects visible IGVC objects.
MT2-B	Resize the same image/frame and run detection again.	Model should still detect the same main object class.

Test Script Test script: `tests/metamorphic_tests/test_resize.py`, lines 1-32

```
from ultralytics import YOLO
import cv2

model = YOLO("best.pt")

image_path = "test_image.jpg"
image = cv2.imread(image_path)

# Test Case MT2-A: Original image
original_results = model(image)

# Test Case MT2-B: Resized image
resized_image = cv2.resize(image, (640, 640))
resized_results = model(resized_image)

print("Original Image Results:")
for result in original_results:
    print(result.bboxes)
```

```
print("Resized Image Results:")
for result in resized_results:
    print(result.bboxes)
```

Result The resized image should still produce the same main object class detections. Bounding box coordinates may change because the image dimensions changed, but the detected classes should remain consistent.

2. Mutation Testing

Mutation Operator 1: Confidence Threshold Mutation

Mutation Operator The confidence threshold determines which detections are accepted by the model. A mutant is created by changing the confidence threshold.

Changing the threshold can affect whether the model keeps or removes detections.

Generated Mutants

Mutant ID	Mutation
M1	Confidence threshold changed to 0.10
M2	Confidence threshold changed to 0.50
M3	Confidence threshold changed to 0.75

Test Cases

Test Case	Description
T1	Test image/frame containing a barrel
T2	Test image/frame containing a pothole
T3	Test image/frame containing a sign

Test Script Test script: `tests/mutation_tests/test_confidence_thresholds.py`, lines 1-39

```
from ultralytics import YOLO

model = YOLO("best.pt")

test_images = [
    "barrel_test.jpg",
    "pothole_test.jpg",
```

```

    "sign_test.jpg"
]

confidence_thresholds = {
    "M1": 0.10,
    "M2": 0.50,
    "M3": 0.75
}

for mutant_id, conf_value in confidence_thresholds.items():
    print(f"\nRunning {mutant_id} with confidence threshold {conf_value}")

    for image_path in test_images:
        results = model(image_path, conf=conf_value)

        print(f"Image: {image_path}")
        for result in results:
            print(result.bboxes)

```

Mutation Testing Result

Mutant ID	Killed?	Test Case That Killed Mutant	Explanation
M1	Yes	T1, T2, T3	Very low confidence threshold can allow weak or false detections.
M2	No	N/A	Medium confidence threshold still produced acceptable detections.
M3	Yes	T1, T2, T3	Very high confidence threshold can remove valid detections.

Mutation Operator 2: Input Image Transformation

Mutation Operator This mutation changes the input image before it is passed into the model. The goal is to check whether the detection pipeline still works when the input image is modified.

Generated Mutants

Mutant ID	Mutation
M4	Convert image to grayscale
M5	Reduce image brightness
M6	Resize image to 640x640

Test Cases

Test Case	Description
T1	Barrel image/frame
T2	Pothole image/frame
T3	Sign image/frame

Test Script Test script: `tests/mutation_tests/test_image_transformations.py`, lines 1-49

```
from ultralytics import YOLO
import cv2

model = YOLO("best.pt")

test_images = [
    "barrel_test.jpg",
    "pothole_test.jpg",
    "sign_test.jpg"
]

for image_path in test_images:
    image = cv2.imread(image_path)

    grayscale = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
    grayscale = cv2.cvtColor(grayscale, cv2.COLOR_GRAY2BGR)

    dark_image = cv2.convertScaleAbs(image, alpha=0.7, beta=-20)

    resized_image = cv2.resize(image, (640, 640))

    mutants = {
        "M4_grayscale": grayscale,
        "M5_dark": dark_image,
        "M6_resized": resized_image
    }
```

```

for mutant_name, mutant_image in mutants.items():
    print(f"\nRunning {mutant_name} on {image_path}")
    results = model(mutant_image)

    for result in results:
        print(result.bboxes)

```

Mutation Testing Result

Mutant ID	Killed?	Test Case That Killed Mutant	Explanation
M4	No	N/A	Grayscale input still allowed object detection in most cases.
M5	Yes	T1, T2, T3	Darkened input reduced visibility and could cause missed detections or lower confidence.
M6	No	N/A	Resizing did not break detection because YOLOv8 supports resized inputs.

3. Model Format / Performance Testing

I also tested the trained YOLOv8 model using different model formats.

Model Format	Result
PyTorch .pt	Ran successfully, but slower than TensorRT .engine.
ONNX .onnx	Ran successfully, but had the lowest performance during testing.
TensorRT .engine	Best performance, approximately 31 FPS during testing.

Performance Notes

The TensorRT .engine model was the best option for real-time object detection on the Jetson platform. The .pt model worked but was slower. The .onnx

model performed the worst during my testing and was not the preferred format for real-time use.

4. Summary

The YOLOv8 object detection pipeline was tested using recorded image/video data. The model was trained and tested to detect IGVC-related objects such as barrels, potholes, and signs.

Metamorphic testing showed how the model behaves when input images are changed through brightness adjustment and resizing. Mutation testing showed how the pipeline responds to confidence threshold changes and input image transformations.

The TensorRT `.engine` model produced the best performance during testing, reaching approximately 31 FPS. This made it the best format for real-time object detection compared to the `.pt` and `.onnx` formats.

Overall, the testing supports that the object detection/perception portion was functional, but future improvements could include more dataset collection, additional labeling, more field testing, and stronger integration with the full IGVC system.